CARBON: Bandwidth-Constrained Synchronisation for Collaborative Notes

Project report

October 2026

Abstract

CARBON is a collaborative plain-text notebook and systems-optimisation experiment. It selects a CRDT update-batching timer that keeps remote edits responsive under a traffic budget. The system uses actual Yjs update sizes, a FIFO relay model, calibration/validation separation, and a convergence invariant. Transient awareness powers coloured remote carets without entering the batching measurements.

1 Problem and scope

A CRDT makes concurrent replicas converge after updates arrive; it does not decide when a local update leaves the browser. Sending every keystroke pays a fixed framing cost. Batching amortises that cost and may decrease relay queueing, but delays edits. CARBON has N replicas and chooses one timer T. The timer starts at the first pending edit and never resets. $T = 0$ is immediate sending; the 16 KiB cap is a fixed safety guard. Reliable delivery and final state equality are hard correctness requirements, not objective terms.

2 Model

Scenario ω contains an edit trace and a relay profile. Edit j appears at t_j and has byte size s_j. Batch b opens at a_b and flushes at

$$f_b = \min\left(a_b + T, \inf\left\{t \geq a_b : \sum_{j:a_b \leq t_j \leq t} s_j \geq S\right\}\right). \tag{1}$$

A run-length approximation is $P_b = s_0 r_b + s_1 m_b$, with m_b edits and r_b contiguous runs. The implementation instead computes P_b from the actual `Y.mergeUpdates` result.

Let H be frame overhead, R FIFO capacity in bytes/s and d propagation delay. With batches in flush order,

$$\sigma_b = (H + P_b)/R, \tag{2}$$

$$W_1 = 0, \quad W_{b+1} = \max\{0, W_b + \sigma_b - (f_{b+1} - f_b)\}. \tag{3}$$

Remote-visible delay is

$$D_j(T; \omega) = (f_b - t_j) + W_b + \sigma_b + d. \tag{4}$$

The components are batching, queueing, serialisation and propagation. The first rises with T; the first two communication terms may fall as batches get larger.

3 Constrained finite optimisation

Let $L_{0.95}(T;\omega)$ be empirical remote-edit p95. With relay fan-out ϕ and fixed input duration Θ, offered traffic is

$$r(T;\omega) = \frac{\phi \sum_b (H + P_b)}{\Theta/1000} \quad \text{bytes/s.} \tag{5}$$

This counts post-trace flushing; capacity-capped output would conceal overload. CARBON enumerates $\mathcal{T} = \{0, 10, \ldots, 300\}$ ms:

$$\min_{T \in \mathcal{T}} \quad \max_{\omega \in \Omega_{\text{train}}} L_{0.95}(T;\omega) \tag{6}$$

$$\text{s.t.} \quad r(T;\omega) \le B, \quad L_{0.95}(T;\omega) \le L_{\max}, \tag{7}$$

$$C(T;\omega) = 1 \qquad \forall \omega \in \Omega_{\text{train}}. \tag{8}$$

C asserts final replica convergence. B is 80% of configured capacity, $L_{\max} = 500$ ms, and ties prefer lower traffic then smaller T. Held-out validation never influences selection.

The wider formulation has $x = (T, S)$ and

$$J(x) = K^{-1} \sum_i \left[w_b B(x;\omega_i)/B_0(\omega_i) + w_l L_{0.95}(x;\omega_i)/L_{\text{SLO}} \right]. \tag{9}$$

Fixing S makes CARBON's enumeration exact over its stated alternatives and more understandable than an unnecessary heuristic.

4 Analytic reference

For one Poisson sender of rate λ, negligible queueing, fixed cost κ and marginal cost $\tilde{s}$,

$$\text{messageRate}(T) = \frac{\lambda}{1 + \lambda T}, \qquad \beta(T) = \phi \left[\lambda \tilde{s} + \frac{\lambda \kappa}{1 + \lambda T} \right]. \tag{10}$$

Thus

$$\frac{\beta(T)}{\beta(0)} = \frac{\tilde{s} + \kappa/(1 + \lambda T)}{\kappa + \tilde{s}}. \tag{11}$$

At low rate $L_{0.95}(T) \approx T + c_{\text{net}}$ and the weighted reference is

$$T^*_{\text{unc}} = \frac{\left[\sqrt{\frac{w_b \lambda \kappa L_{\text{SLO}}}{w_l (\kappa + \tilde{s})}} - 1 \right]_+}{\lambda}. \tag{12}$$

The constrained reference takes the minimum of this, $L_{\text{SLO}} - c_{\text{net}}$ and $T_{\max}$. It validates the light-load simulator but does not claim to optimise bursty multi-user traces.

5 Algorithm and architecture

For every timer candidate, the browser generates or captures identical calibration/validation Yjs traces, batches actual updates, and checks a quiescent replica against the unbatched reference. Python replays batches through FIFO egress once per remote recipient and returns p95, rate, message count, maximum queue, final drain, feasibility and convergence. The selector rejects infeasible candidates and chooses the lexicographically best calibration result.

React/Vite/TypeScript renders CodeMirror bound to Yjs. FastAPI relays binary document updates and persists independent coalesced groups to MongoDB or SQLite. Reconnecting clients receive history, then return merged local state; Yjs makes duplicate update delivery safe.

A separate Yjs Awareness channel carries a name, deterministic colour and cursor/selection state. FastAPI relays it only while a session exists; it is neither persisted nor included in the experiment. The matching avatar, labelled caret and selection colour make simultaneous editing unmistakable in a demonstration.

Table 1: Controlled measurement boundary.

Included	Excluded
Merged Yjs payload bytes	Client upload and initial sync
FIFO relay, fan-out, frame size	Awareness/presence traffic
Queue, propagation, final drain	TCP/TLS/IP and retransmission
Final CRDT convergence	Packet-level wire-byte claims

6 Evaluation protocol

Sample typing uses deterministic steady and burst/paste traces. Recorded typing is split into calibration and validation and stops if a remote update arrives, since asynchronous causal timing is not inferred by this simple capture model. Every candidate sees equal trace inputs. We compare immediate sending, a fixed 100 ms baseline and the selected timer. A no-feasible-candidate result is reported honestly rather than replaced with a decorative optimum.

7 Reproduction, deployment and limitations

On Linux, install Node.js 22+ and Python 3.11+, run `npm ci`, create a virtual environment, install `backend/requirements.txt`, and run the frontend, backend and build tests listed in the README. Two browser windows must open the same room link to show convergence and remote carets.

The repository's multi-stage Dockerfile builds the Vite frontend and serves it through FastAPI. One Render Web Service can host static client, API and WebSocket on one HTTPS origin; the browser therefore uses `wss`. A hosted MongoDB URI is needed for durable public data. One service instance is required because live rooms and awareness are in memory.

The FIFO model has fixed capacity and propagation, not time-varying loss, browser scheduling, multi-hop routing or packet-level accounting. Replay p95 is not observed one-way internet latency. These stated limits keep the contribution reproducible: a real collaborative editor plus an exact finite search over a controlled, clearly bounded model.

A Demonstration checklist and reproducibility contract

Before demonstrating CARBON, start the single backend/frontend service and confirm its health endpoint. Open a room in two independently named browser windows, place their cursors on separate lines, and type concurrently. The two labelled colours should be visible immediately, while the final text converges in both windows. The experiment should then be shown separately: select a network profile, run the fixed 31-value search, and explain that its selected timer is derived from recorded or deterministic input traces rather than from the coloured caret messages.

The reproducibility contract is: each candidate receives the same trace structure; CRDT payload sizes come from the same Yjs implementation; every FIFO queue drains; and held-out traces cannot influence the choice. Any deployment report should state whether it used the disposable SQLite fallback or hosted MongoDB, the public service's single-instance constraint, and the profile parameters used in the experiment. These details prevent a presentation from overstating a controlled simulation as an unrestricted internet-performance result.

References

[1] K. Kevin Jahns, *Yjs document updates*, `https://docs.yjs.dev/api/document-updates`.

[2] S. Ramírez, *FastAPI WebSockets*, `https://fastapi.tiangolo.com/advanced/websockets/`.

[3] I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455, 2011.